Manchester Metropolitan University

Week 6

UI Testing

Dr Lewis Evans

OUTLINE

- UI Testing
- UI Testing Tools
- TestFX (tool for testing JavaFX applications)

UI TESTING

- **UI Testing**: “Verifying the graphical interface of an application”
  - Ensuring that UI elements and user interactions lead to expected results
- Testing UI components ensures a positive user experience by detecting and preventing UI-related issues.

CHALLENGES IN UI TESTING

Variety of Devices and Platforms:

- Testing may cover multiple devices, OS versions, and screen sizes.

Dynamic and Evolving UIs:

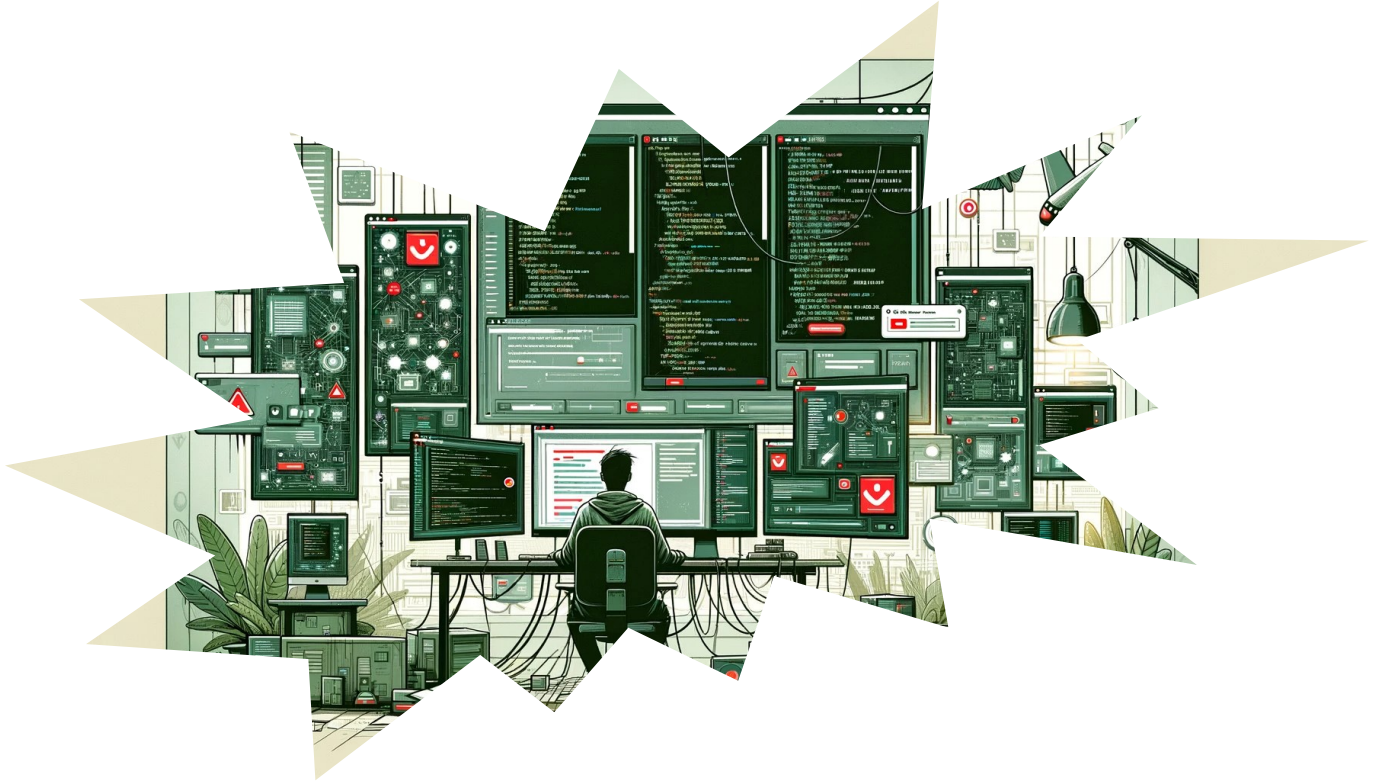
- UIs undergo frequent changes and updates, making testing challenging.

Data-Driven Testing:

- Handling various data inputs and user scenarios is crucial in UI testing.

Test Automation Complexity:

- UI test automation is more complex compared to other testing levels due to GUI interactions.



CONTEXT-DEPENDENT UI TESTING TOOLS

- I've harped on about this a few times; **testing is context-dependent**
- One tool may be perfectly equipped to perform UI testing on a website, but be unsuitable for other applications

Tool	Common Environment
Selenium	Web environments
XCUITest	iOS applications
Espresso	Android applications
Appium	Mobile applications (iOS, Android)
WinAppDriver	Windows desktop applications
Protractor	Angular web applications
TestFX	JavaFX desktop applications

TESTFX

- TestFX is an **open-source framework** for **automated UI testing** of JavaFX applications.
- It provides a **set of tools and libraries for writing tests** that interact with and verify the behaviour of JavaFX user interfaces.
- TestFX **simplifies UI testing by allowing you to automate user interactions** and perform assertions on UI components programmatically.
- TestFX works with the **JavaFX scene graph**, which represents the UI components and their hierarchy.
- It enables **interactions with UI components** like buttons, text fields, and checkboxes.
- TestFX supports **assertions and verifications** to ensure that UI elements behave as expected.

TESTFX

- Let’s look at a very simple example of a JavaFX application and see how TestFX can automate the moving of the mouse and interact with a simple component (button)
- We’ll then increase this in complexity (particularly in the labs)

SETTING UP TESTFX

- If using a build system such as Maven, we can add TestFX (and also JUnit) dependencies to our **pom.xml** file

```
<dependencies>
    <!-- JUnit 5 -->
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-api</artifactId>
        <version>5.7.2</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.7.2</version>
        <scope>test</scope>
    </dependency>

    <!-- TestFX -->
    <dependency>
        <groupId>org.testfx</groupId>
        <artifactId>testfx-core</artifactId>
        <version>4.0.16-alpha</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.testfx</groupId>
        <artifactId>testfx-junit5</artifactId>
        <version>4.0.16-alpha</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

TESTFX DEMO SETUP – FXML FILE

- As you may have seen in the JavaFX example in the lab last week, one common approach to structuring a JavaFX application is having an FXML file to *describe* what our GUI looks like

SimpleApp.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<StackPane prefHeight="200.0" prefWidth="300.0" xmlns="http://javafx.com/javafx/17.0.2-ea"
           xmlns:fx="http://javafx.com/fxml/1" fx:controller="SimpleAppController">
    <Button fx:id="clickMeButton" onAction="#handleButtonClick" text="Click Me!" />
</StackPane>
```

TESTFX DEMO SETUP – FXML FILE

- We also typically have a Controller class that takes care of interactions

SimpleAppController.java

SimpleApp.fxml | SimpleAppController.java

```java
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.scene.control.Button;

public class SimpleAppController {
    @FXML
    private Button clickMeButton;

    @FXML
    private void handleButtonClick(ActionEvent event) {
        clickMeButton.setText("Clicked!");
    }
}
```

TESTFX DEMO SETUP – ‘MAIN’ CLASS

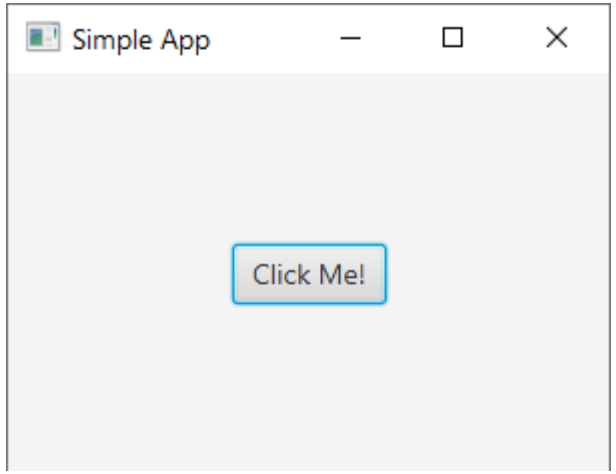
The “main” class that launches the application

SimpleApp.java

```java
import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class SimpleApp extends Application {
    public static void main(String[] args) {
        launch(args);
    }

    @Override
    public void start(Stage primaryStage) throws Exception {
        FXMLLoader loader = new FXMLLoader(getClass().getResource( name: "SimpleApp.fxml"));
        Parent root = loader.load();
        SimpleAppController controller = loader.getController();

        primaryStage.setTitle("Simple App");
        primaryStage.setScene(new Scene(root,  width: 300,  height: 200));
        primaryStage.show();
    }
}
```

TESTFX DEMO – APP RUNNING

- We could of course just click on this button and visually observe the text changes (that would of course be a manual test)
- At this point, we have not used TestFX – let's take a look at how this would work



TESTFX DEMO

start Method

- Launches the JavaFX application by calling the start() method of the SimpleApp class (Initialises the JavaFX application for testing)

testButtonClick Method

- This is a JUnit test method annotated with @Test.
- It defines a test scenario for clicking a button in the UI.
- Inside the method, the clickOn("#clickMeButton") statement simulates a click on a UI element with the given ID.
- The **verifyThat("#clickMeButton", hasText("Clicked!"))** statement verifies that after clicking the button, its text changes to "Clicked!" using TestFX assertions.

stop Method

- This method is inherited from ApplicationTest and is called when the test ends.
- Hides the JavaFX stage, ensuring that any remaining UI elements are closed properly.
- Performs cleanup after the test execution.

```java
import javafx.stage.Stage;
import org.junit.jupiter.api.Test;
import org.testfx.api.FxToolkit;
import org.testfx.framework.junit5.ApplicationTest;
import static org.testfx.api.FxAssert.verifyThat;
import static org.testfx.matcher.control.LabeledMatchers.hasText;

public class SimpleAppTest extends ApplicationTest {

    @Override
    public void start(Stage stage) throws Exception {
        new SimpleApp().start(stage);
    }

    @Test
    public void testButtonClick() {
        // Click the button
        clickOn( query: "#clickMeButton");

        // Verify that the button text changes to "Clicked!"
        verifyThat( nodeQuery: "#clickMeButton", hasText("Clicked!"));
    }

    @Override
    public void stop() throws Exception {
        FxToolkit.hideStage();
    }
}
```

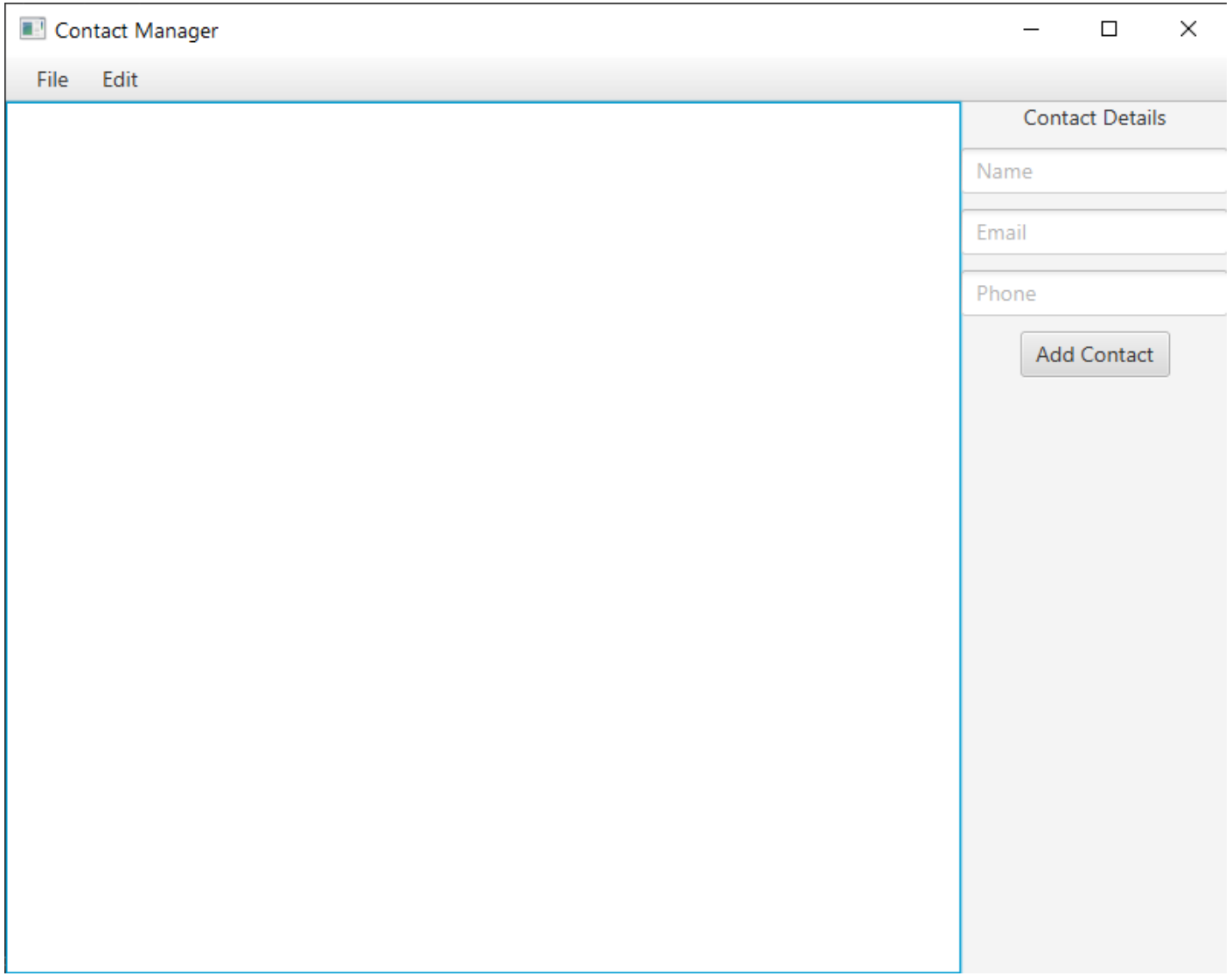
OUTPUT

```java
public class SimpleAppTest extends ApplicationTest {

    @Override
    public void start(Stage stage) throws Exception {
        new SimpleApp().start(stage);
    }

    @Test
    public void testButtonClick() {
        // Click the button
        clickOn( query: "#clickMeButton");

        // Verify that the button text changes to "Clicked!"
        verifyThat( nodeQuery: "#clickMeButton", hasText("Clicked!"));
    }

    @Override
    public void stop() throws Exception {
        FxToolkit.hideStage();
    }
}
```

```
C:\Users\55116318\.jdks\corretto-1.8.0_382\bin\java.exe ...

Process finished with exit code 0
```

TESTFX EXAMPLE 2: CONTACT MANAGER APP

- Simple contact manager application
- We want to write a test that when the user enters the three pieces of information, and click **Add Contact**, it causes the ListView to update with the added contact



TESTFX EXAMPLE 2: CONTACT MANAGER APP

- I'll add the full source code for this onto Moodle. But for now let's focus on the Test itself

TESTFX EXAMPLE 2: CONTACT MANAGER APP

- I'll add the full source code for this onto Moodle. But for now let's focus on the Test itself

```java
15      @Test
16      public void testAddContact_ShouldAddContactToGUI() {
17          // Click on the "Name" field and enter a name
18          clickOn( query: "#nameField").write( s: "John Doe");
19
20          // Click on the "Email" field and enter an email
21          clickOn( query: "#emailField").write( s: "john@example.com");
22
23          // Click on the "Phone" field and enter a phone number
24          clickOn( query: "#phoneField").write( s: "123-456-7890");
25
26          // Click on the "Add Contact" button
27          clickOn( query: "#addContact");
28
29          // Verify that the ListView contains the entered contact information
30          ListView<String> contactListView = lookup( query: "#contactListView").query();
31          verifyThat(contactListView, hasListCell( value: "John Doe | john@example.com | 123-456-7890"));
32
33          // Sleep for 1 second (just so we can see the thing being added)
34          sleep( milliseconds: 1000);
35      }
```

File Edit View Navigate Code Refactor Build Run Tools VCS Window Help TestFXLectureDemo2 - ContactManagerAppTest.java

TestFXLectureDemo2 › src › test › java › ContactManagerAppTest › testAddContact_ShouldAddContactToGUI

ContactManagerAppTest.testAddContact_ShouldAddContactToGUI

pom.xml (TestFXLectureDemo) | ContactManager.fxml | ContactManagerApp.java | ContactManagerAppTest.java | ContactManagerController.java

```
public class ContactManagerAppTest extends ApplicationTest {

    @Override
    public void start(Stage stage) throws Exception {
        new ContactManagerApp().start(stage);
    }

    @Test
    public void testAddContact_ShouldAddContactToGUI() {
        // Click on the "Name" field and enter a name
        clickOn( query: "#nameField").write( s: "John Doe");

        // Click on the "Email" field and enter an email
        clickOn( query: "#emailField").write( s: "john@example.com");

        // Click on the "Phone" field and enter a phone number
        clickOn( query: "#phoneField").write( s: "123-456-7890");

        // Click on the "Add Contact" button
        clickOn( query: "#addContact");

        // Verify that the ListView contains the entered contact information
        ListView<String> contactListView = lookup( query: "#contactListView").query();
        verifyThat(contactListView, hasListCell( value: "John Doe | john@example.com | 123-456-7890"));

        // Sleep for 1 second (just so we can see the thing being added)
        sleep( milliseconds: 1000);
    }

}
```

Project

- TestFXLectureDemo2 C:\Users\55116318\Desktop\TestFXLectureDemo2
  - .idea
    - .gitignore
    - compiler.xml
    - encodings.xml
    - jarRepositories.xml
    - misc.xml
    - workspace.xml
  - src
    - main
      - java
        - ContactManagerApp
        - ContactManagerController
      - resources
        - ContactManager.fxml
    - test
      - java
        - ContactManagerAppTest
  - target
  - pom.xml
- External Libraries
- Scratches and Consoles

Run: ContactManagerAppTest.testAddContact_ShouldAddContactToGUI

Tests passed: 1 of 1 test – 6 sec 678 ms

- ContactManagerAppTest 6 sec 678 ms
  - testAddContact_ShouldAddContactToGUI() 6 sec 678 ms

```
C:\Users\55116318\.jdks\corretto-1.8.0_382\bin\java.exe ...

Process finished with exit code 0
```

Version Control | Run | TODO | Problems | Terminal | Services | Build | Dependencies

Tests passed: 1 (a minute ago) 26:7 CRLF UTF-8 4 spaces

SUMMARY

- We've taken a look at UI Testing today, including how we can automate it for the purposes of the second assessment
- TestFX – used to automate testing of UI components in a JavaFX application
- The labs will take this further

Lab A – Intro to TestFX	Lab B – Intermediate TestFX
The first lab will give us a gentle introduction to TestFX with some simple GUI components.	The second lab will add a little more complexity to things in terms of how we can test other types of components.